

力扣 Hot 100 必玩项目

晨读背诵吟唱内容

官方分类顺序+每题含题目描述+示例

题与题之间已留出空白，方便吟唱

有人相爱，有人开车看海，有人一天做不出一道题

一、哈希

本类共 3 道题

1. 两数之和 【简单】

给定一个整数数组 `nums` 和一个整数目标值 `target`，请你在该数组中找出和为目标值 `target` 的那两个整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。你可以按任意顺序返回答案。

示例 1：

输入：`nums = [2,7,11,15]`, `target = 9`

输出：`[0,1]`

解释：因为 `nums[0] + nums[1] == 9`，返回 `[0, 1]`。

示例 2：

输入：`nums = [3,2,4]`, `target = 6`

输出：`[1,2]`

💡 学霸笔记：

两层循环暴力，我没意见，回去等通知写法。

正经点哈希表法，用 `int`，`int` 的 `map` 存 `key` 是 `nums[i]`, `value` 是 `i`，开 `for` 遍历一遍 `nums`，`if` 找 `map` 的 `key` 为 `target-nums[i]`，找到 `return` 没找到加入新的。最后随便 `return` 个啥反正也走不到，对了 `Map` 不会漏原因是两数之和必定是两个数，所以先塞进去必定会等后面的。结束战斗

```

class Solution {
    public int[] twoSum(int[] nums, int target) {
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[0];
    }
}

```

```

class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer,Integer> map = new HashMap<>();
        for(int i = 0;i < nums.length;i++){
            int need = target - nums[i];
            if(map.containsKey(need)){
                return new int[]{map.get(need),i};
            }else{
                map.put(nums[i],i);
            }
        }
        return nums;
    }
}

```

49. 字母异位词分组 【中等】

给你一个字符串数组，请你将字母异位词组合在一起。可以按任意顺序返回结果列表。

字母异位词是由重新排列源单词的所有字母得到的一个新单词。

示例 1：

输入: strs = ["eat", "tea", "tan", "ate", "nat", "bat"]

输出: [["bat"],["nat","tan"],["ate","eat","tea"]]

输入: strs = [""]

输出: [[""]]

📌 学霸笔记：

开个 map 存，key 是 str ， value 是要求的内容。For 循环 strs，里面 new 一个排序版 str，这样好判断 map.containsKey(key)，没有就加进去，最后 return 构造新的 list，内容是 map.values()

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String,List<String>> map = new HashMap<>();
        for(String i : strs){
            char[] c = i.toCharArray();
            Arrays.sort(c);
            String key = new String(c);

            if(!map.containsKey(key)){
                map.put(key,new ArrayList<>());
            }
            map.get(key).add(i);
        }
        return new ArrayList<> (map.values());
    }
}
```

128. 最长连续序列 【中等】

给定一个未排序的整数数组 nums ，找出数字连续的最长序列（不要求序列元素在原数组中连续）的长度。

请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例 1:

输入: nums = [100,4,200,1,3,2]

输出: 4

解释: 最长数字连续序列是 [1, 2, 3, 4]。它的长度为 4。

示例 2:

输入：nums = [0,3,7,2,5,8,4,6,0,1]

输出：9

📖 学霸笔记：

用 set (hashset) ，for 加进去 nums 到 set，开两层循环，外面 set，判断 contains(x-1)，里面开 while (y =x+1) 用来找下一个，y++，退到外层 Math.max 记录一下最大值 (y-x,原) ，结束战斗

```
class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> st = new HashSet<>();
        for(int num : nums){
            st.add(num);
        }
        int ans = 0;
        for(int x :st){
            if(st.contains(x - 1)){
                continue;
            }
            int y = x + 1;
            while(st.contains(y)){
                y++;
            }
            ans = Math.max(ans,y - x);
        }
        return ans;
    }
}
```

二、双指针

本类共 4 道题

283. 移动零 【简单】

给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。

请注意，必须在不复制数组的情况下原地对数组进行操作。

示例 1:

输入: `nums = [0,1,0,3,12]`

输出: `[1,3,12,0,0]`

示例 2:

输入: `nums = [0]`

输出: `[0]`

📌 学霸笔记:

变量 `j` 0，第一次 `for i-n`，判断是不是 0，不是就 `num[i]` 赋给 `num[j]` 并 `j++`，开第二次 `for(j)-n`，此时 `num` 都应该赋 0，结束战斗

```
class Solution {
    public void moveZeroes(int[] nums) {
        if(nums==null) {
            return;
        }
        int j = 0;
        for(int i=0;i<nums.length;++i) {
            if(nums[i]!=0) {
                nums[j++] = nums[i];
            }
        }
        for(int i=j;i<nums.length;++i) {
            nums[i] = 0;
        }
    }
}
```

11. 盛最多水的容器 【中等】

给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, \text{height}[i])$ 。找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。返回容器可以储存的最大水量。

示例 1:

输入: `[1,8,6,2,5,4,8,3,7]`

输出: 49

解释: 图中垂直线代表输入数组 `[1,8,6,2,5,4,8,3,7]`。在此情况下，容器能够容纳水（表示为蓝色部分）的最大值为 49。

💡 学霸笔记:

双指针，开 `while(i 头 j 尾)`，`math` 记录最大值面积，面积取决最短高度，`i` 的边比较 `j` 的边，谁短谁移动(`i++j--`)，最后 `return` 结束战斗

```
class Solution {
    public int maxArea(int[] height) {
        int result = 0;
        int area = 0;
        int j = height.length - 1;
        int i = 0;
        while(i < j){
            area = (j-i) * Math.min(height[i],height[j]);
            result = Math.max(area,result);
            if(height[i] < height[j]){
                i++;
            }
            else{
                j--;
            }
        }
        return result;
    }
}
```

15. 三数之和 【中等】

给你一个整数数组 `nums`，判断是否存在三元组 `[nums[i], nums[j], nums[k]]` 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足 $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组。注意：答案中不可以包含重复的三元组。

示例 1:

输入: `nums = [-1,0,1,2,-1,-4]`

输出: `[[-1,-1,2],[-1,0,1]]`

示例 2:

输入: `nums = [0,1,1]`

输出: `[]`

💡 学霸笔记:

定义 `List<List<` 的 `result`, `sort` 下方便剪枝，开两层循环，外面 `for i-n`，剪枝(`if i` 和 -1 一样就 `continue` 保证不重复)和定义 `left(i 右)` 和 `right(最右)`，里面 `while` 用双指针，判断有无 `+++=0` 加入 `result`，`while` 剪枝(左右往中间靠)，然后正常 `j++k--`(加 `result` 都移，没加移一个).退循环 `return` 结束战斗


```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        List<List<Integer>> result = new ArrayList<>();
        Arrays.sort(nums);
        for(int i=0;i<nums.length;i++){
            if(i>0 && nums[i] == nums[i-1]) continue;
            int left = i+1;
            int right = nums.length - 1;
            while(left<right){
                if( nums[left] + nums[right] +nums[i] == 0){
                    result.add(Arrays.asList(nums[i],nums[left],nums[right]));
                    while (right > left && nums[right] == nums[right - 1]) right--;
                    while (right > left && nums[left] == nums[left + 1]) left++;
                    right--;
                    left++;
                }else if( nums[left] + nums[right] +nums[i] < 0){
                    left++;
                }else{
                    right--;
                }
            }
        }
        return result;
    }
}

```

42. 接雨水 【困难】

给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

示例 1：

输入：height = [0,1,0,2,1,0,1,3,2,1,2,1]

输出：6

解释：上面是由数组 [0,1,0,2,1,0,1,3,2,1,2,1] 表示的高度图，在这种情况下，可以接 6 个单位的雨水（蓝色部分表示雨水）。

示例 2：

输入：height = [4,2,0,3,2,5]

输出：9

📌 学霸笔记：

可用双指针，装逼可用前缀和，但是难背，弃了

定义左(0)右(边界),左 max0 右 max0，开一层循环 while 来双指针，判断左右谁矮，矮的一边再开 if 判断高度和各自 max 比较，>=就更新 max，小就接一波雨水+=max-矮边，最后指针往中间移，return 面积，结束战斗

```
class Solution {
    public int trap(int[] height) {
        int left = 0;
        int right = height.length - 1;
        int leftmax = 0 , rightmax = 0 ,water = 0;
        while(left < right){
            if(height[left] < height[right]){
                if(height[left] >= leftmax){
                    leftmax = height[left];
                }
                else{
                    water += leftmax - height[left];
                }
                left++;
            }
            else{
                if(height[right] >=rightmax){
                    rightmax = height[right];
                }
                else{
                    water += rightmax - height[right];
                }
                right--;
            }
        }
        return water;
    }
}
```

三、滑动窗口

本类共 4 道题

3. 无重复字符的最长子串 【中等】

给定一个字符串 s ，请你找出其中不含有重复字符的 最长子串 的长度。

示例 1:

输入: $s = \text{"abcabcbb"}$

输出: 3

解释: 因为无重复字符的最长子串是 "abc"，所以其长度为 3。

示例 2:

输入: $s = \text{"bbbbbb"}$

输出: 1

解释: 因为无重复字符的最长子串是 "b"，所以其长度为 1。

📖 学霸笔记:

定义左 0 右边界，哈希 $\text{int}[]$ count 存字母出现次数，开一个 for i-length，里面用 c 转一下 char 存到 $\text{count}[c]++$ ，进 while ($\text{count}[c]>1$) 来把左移到不重复， $\text{count}[s[\text{left}]]--$ ，最后 $\text{left}++$ 退 while，Math 记一下最长的值 $i-\text{left}+1$ ，return 结束战斗

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        char[] S = s.toCharArray();
        int left = 0;
        int result = 0;
        int right = s.length();
        int[] cnt = new int[128];
        for(int i = 0; i < right; i++){
            char c = S[i];
            cnt[c]++;
            while(cnt[c] > 1){
                cnt[S[left]]--;
                left++;
            }
            result = Math.max(result, i - left + 1);
        }
        return result;
    }
}
```

438. 找到字符串中所有字母异位词 【中等】

给定两个字符串 s 和 p ，找到 s 中所有 p 的异位词 的子串，返回这些子串的起始索引。不考虑答案输出的顺序。

异位词 指由相同字母重排列形成的字符串（包括相同的字符串）。

示例 1：

输入: $s = \text{"cbaebabacd"}, p = \text{"abc"}$

输出: $[0,6]$

解释:

起始索引等于 0 的子串是 "cba", 它是 "abc" 的异位词。

起始索引等于 6 的子串是 "bac", 它是 "abc" 的异位词。

示例 2:

输入: $s = \text{"abab"}, p = \text{"ab"}$

输出: $[0,1,2]$

📖 学霸笔记:

定义 s 、 p count 频次作为哈希表查，判断 p length 是不是大于 s ，开两次 for，第一次 for 统计 $pcount[i]++$ ，退 for 第二次 for $i-s$ ，还是先统计，后 if $i \geq 0$ (0 开始所以 $=$) 窗口删除 $scount[i - 窗口]--$ ，if $scount$ 和 $pcount$ 相等就加 result，return 结束战斗

```

class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> result = new ArrayList<>();
        if (s.length() < p.length()) return result;
        int[] pCount = new int[128];
        int[] sCount = new int[128];
        // 统计 p 的字符频率
        for (char c : p.toCharArray()) {
            pCount[c]++;
        }
        int windowSize = p.length();
        for (int i = 0; i < s.length(); i++) {
            // 加入新字符
            sCount[s.charAt(i)]++;
            // 移除超出窗口的字符
            if (i >= windowSize) {
                sCount[s.charAt(i - windowSize)]--;
            }
            // 比较当前窗口
            if (Arrays.equals(sCount, pCount)) {
                result.add(i - windowSize + 1); // 添加左边界
            }
        }
        return result;
    }
}

```

560. 和为 K 的子数组 【中等】

给你一个整数数组 `nums` 和一个整数 `k`，请你统计并返回 该数组中和为 `k` 的连续子数组的个数。

子数组是数组中元素的连续非空序列。

示例 1：

输入：`nums = [1,1,1], k = 2`

输出：`2`

示例 2：

输入：`nums = [1,2,3], k = 3`

输出：2

📖 学霸笔记：

因为要找连续的子数组，定义前缀和 $s[]$ 存 sn ，map 值存次数，key 是 sn ，开 for 定义一遍 $sn=s[n-1]+a[n-1]$ ，退 for 开新 for sn ，里面先 $result +=$ 找 key 是 $sn-k$ 的 value，再 if 这次遍历的 sn 不在 map 里，存 1 否则 +1。return 结束战斗

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int n = nums.length;
        int[] s = new int[n+1];
        int result = 0;
        for(int i = 0; i < n; i++){
            s[i+1] = s[i] + nums[i];
        }
        Map<Integer,Integer> map = new HashMap<>();
        for(int js: s){
            result += map.getDefault(js-k,0);
            // map.merge(js,1,Integer::sum);
            if(!map.containsKey(js)) map.put(js,1);
            else{
                map.put(js,map.get(js)+1);
            }
        }
        return result;
    }
}
```

239. 滑动窗口最大值 【困难】

给你一个整数数组 $nums$ ，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只能看到在滑动窗口内的 k 个数字。滑动窗口每次只向右移动一位。

返回 滑动窗口中的最大值。

示例 1：

输入: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`

输出: `[3,3,5,5,6,7]`

解释:

滑动窗口的位置 最大值

```
-----
[1 3 -1] -3 5 3 6 7      3
1 [3 -1 -3] 5 3 6 7      3
1 3 [-1 -3 5] 3 6 7      5
1 3 -1 [-3 5 3] 6 7      5
1 3 -1 -3 [5 3 6] 7      6
1 3 -1 -3 5 [3 6 7]      7
```

📖 学霸笔记:

维护一个双端队列 `q`, for (`i`(窗口最左), `j`0,`n`-窗口), 里面先 if 判断 `i` 大于 0 表示队列满了, 且队列头 == `num i-1` 不在窗口了, 该划走; 继续新人入队开个 while 清理比他小的都 remove; 清理完了退 while 加自己 `numj`, 记录当下队头 `i`(`i` 要 > 0) 进 `result`, return 结束战斗

总结: 队列满清老大 `i`, 队列进新人从后往前 while 踢比自己弱, 自己加到尾, 自己最弱就直接加到尾

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        if(nums.length == 0 || k == 0) return new int[0];
        Deque<Integer> q = new LinkedList<>();
        int[] res = new int[nums.length - k + 1];
        for(int j = 0, i = 1 - k; j < nums.length; i++, j++){
            if(i > 0 && q.peekFirst() == nums[i-1]) q.removeFirst();
            while(!q.isEmpty() && q.peekLast() < nums[j]) q.removeLast();
            q.addLast(nums[j]);
            if(i >= 0){
                res[i] = q.peekFirst();
            }
        }
        return res;
    }
}
```

四、子串

本类共 2 道题

76. 最小覆盖子串 【困难】

给你一个字符串 s 、一个字符串 t 。返回 s 中涵盖 t 所有字符的最小子串。如果 s 中不存在涵盖 t 所有字符的子串，则返回空字符串 `""`。

注意：

- 对于 t 中重复字符，我们寻找的子字符串中该字符数量必须不少于 t 中该字符数量。
- 如果 s 中存在这样的子串，我们保证它是唯一的答案。

示例 1：

输入： $s = \text{"ADOBECODEBANC"}, t = \text{"ABC"}$

输出：`"BANC"`

解释：最小覆盖子串 `"BANC"` 包含来自字符串 t 的 ' A '、' B ' 和 ' C '。

示例 2：

输入： $s = \text{"a"}, t = \text{"a"}$

输出：`"a"`

💡 学霸笔记：

其实可以不背，但还是讲讲思路吧，说不定以后想背了。

定义 $\text{diff}[c]$ 表示 c 这个东西的差异， dept 表示各种 diff 加起来总差异，开 $\text{for } t$ 定义一遍 t 的 diff 东西有多少，退出来开第二个 $\text{for } i-s$ ，判断有没有 $\text{diff}[i]$ ，有就减少个差异，接着开 while ($\text{dept}=0$)作用是更新窗口最小值和尝试缩小左边界，里面判断窗口大小比 len ，小就更新，接着看左边界右移 diff 和 dept 变化，退 while for ,return s 的 sub 串。


```

class Solution {
    public String minWindow(String s, String t) {
        int[] diff = new int[128];
        for (char c : t.toCharArray()) diff[c]++;
        int debt = t.length();
        int left = 0, start = 0, len = Integer.MAX_VALUE;
        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (diff[c]-- > 0) debt--;
            while (debt == 0) {
                if (right - left + 1 < len) {
                    start = left;
                    len = right - left + 1;
                }
                char d = s.charAt(left++);
                if (++diff[d] > 0) debt++;
            }
        }
        return len == Integer.MAX_VALUE ? "" : s.substring(start, start + len);
    }
}

```

139. 单词拆分 【中等】

给你一个字符串 *s* 和一个字符串列表 *wordDict* 作为字典。如果可以利用字典中出现的一个或多个单词拼接出 *s* 则返回 *true*。

注意：不要求字典中出现的单词全部都使用，并且字典中的单词可以重复使用。

示例 1：

输入: *s* = "leetcode", *wordDict* = ["leet", "code"]

输出: **true**

解释: 返回 **true** 因为 "leetcode" 可以由 "leet" 和 "code" 拼接成。

示例 2：

输入: *s* = "applepenapple", *wordDict* = ["apple", "pen"]

输出: **true**

解释: 返回 **true** 因为 "applepenapple" 可以由 "apple" "pen" "apple" 拼接成。

📖 学霸笔记:

背诵: $dp[i]$ 指 s 字符在 i 前面全都可以由 wordDict 找到

开两个 for, 外面 $i-s$, 里面 $j-i$, 判断 $dp[j] \&\& \text{substring}(j,i)$ 就设置 $dp[i] = 1$

直接起飞, 结束战斗

```
class Solution {
    public boolean wordBreak(String s, List<String> wordDict) {
        int[] dp = new int[s.length() + 1];
        dp[0] = 1;
        for(int i = 1; i <= s.length(); i++){
            for(int j = 0; j < i; j++){
                String sub = s.substring(j,i);
                if(wordDict.contains(sub) && dp[j] == 1) dp[i] = 1 ;
            }
        }
        return dp[s.length()] == 0 ? false : true;
    }
}
```

五、普通数组

本类共 5 道题

53. 最大子数组和 【中等】

给你一个整数数组 $nums$, 请你找出一个具有最大和的**连续子数组** (子数组最少包含一个元素) , 返回其最大和。

子数组是数组中的一个连续部分。

示例 1:

输入: $nums = [-2,1,-3,4,-1,2,1,-5,4]$

输出: 6

解释: 连续子数组 $[4,-1,2,1]$ 的和最大, 为 6 。

示例 2:

输入: `nums = [1]`

输出: `1`

📖 学霸笔记:

初始化 `result` 为 `num[0]` 开一层 `for i-length` 里面用 `Math` 记 `numi += 最大(0, i-1)` 表示取不取前一个, 不取正好断了, 用 `result` 记录历史最大 `num`。

```
class Solution{
    public int maxSubArray(int[] nums){
        int result = nums[0];
        if(nums.length == 1) return nums[0];
        for(int i = 1; i < nums.length; i++){
            nums[i] += Math.max(nums[i - 1], 0);
            result = Math.max(result, nums[i]);
        }
        return result;
    }
}
```

56. 合并区间 【中等】

以数组 `intervals` 表示若干个区间的集合, 其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间, 并返回一个不重叠的区间数组, 该数组需恰好覆盖输入中的所有区间。

示例 1:

输入: `intervals = [[1,3],[2,6],[8,10],[15,18]]`

输出: `[[1,6],[8,10],[15,18]]`

解释: 区间 `[1,3]` 和 `[2,6]` 重叠, 将它们合并为 `[1,6]`。

📖 学霸笔记:

把合并区间想成水, 有重叠就融成大区间了, 没重叠就 `new`。

先 sort 排序[]的左边也就是各个区间出发点，升序，定义 res 用 List 包一下后面转，开 for-intervals，里面判断 res 有没有或者不重叠，加区间，有重叠就 Math 取一下结束点最大值，退出 return toArray 转一下结束战斗

```
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
        List<int[]> res = new ArrayList<>();
        for (int[] cur : intervals) {
            int n = res.size();
            if (n == 0 || cur[0] > res.get(n-1)[1]) {
                res.add(cur); // 不重叠 → 加新的
            } else {
                res.get(n-1)[1] = Math.max(res.get(n-1)[1], cur[1]); // 重叠 → 扩 end
            }
        }
        return res.toArray(new int[res.size()][2]);
    }
}
```

189. 轮转数组 【中等】

给定一个整数数组 nums，将数组中的元素向右轮转 k 个位置，其中 k 是非负数。

示例 1：

输入: nums = [1,2,3,4,5,6,7], k = 3

输出: [5,6,7,1,2,3,4]

解释：

向右轮转 1 步: [7,1,2,3,4,5,6]

向右轮转 2 步: [6,7,1,2,3,4,5]

向右轮转 3 步: [5,6,7,1,2,3,4]

示例 2：

输入: nums = [-1,-100,3,99], k = 2

输出: [3,99,-1,-100]

📌 学霸笔记:

定义 k 取余 length 以免超长, 反转函数三次, 一次(0,length-1)、二次(0,k-1)、三次(k,length-1), 反转函数就双指针解决就行。

我说白了这种技巧性的题目就是死记硬背, 和那个环形链表相遇记公式有什么区别, 应试味拉满了, 评论区还妙啊妙啊, 我给到拉完了。

```
class Solution {
    public void rotate(int[] nums, int k) {
        if(nums.length==1){
            return;
        }
        k %= nums.length;
        reverse(nums,0,nums.length-1);
        reverse(nums,0,k-1);
        reverse(nums,k,nums.length-1);
    }

    private void reverse(int[] nums,int start,int end){
        while (start<end){
            int temp = nums[start];
            nums[start] = nums[end];
            nums[end] = temp;
            start++;
            end--;
        }
    }
}
```

238. 除自身以外数组的乘积 【中等】

给你一个整数数组 `nums` , 返回 数组 `answer` , 其中 `answer[i]` 等于 `nums` 中除 `nums[i]` 之外其余各元素的乘积 。

题目数据保证 数组 `nums` 之中任意元素的全部前缀元素和后缀的乘积都在 32 位 整数范围内。

请 不要使用除法，且在 $O(n)$ 时间复杂度内完成此题。

示例 1:

输入: `nums = [1,2,3,4]`

输出: `[24,12,8,6]`

📖 学霸笔记:

这道题想成构造两个 sn, i 前面的 sn 定义为 pre; i 后面的 sn 定义为 suf。三次 for, 一次 for i-length 定义 pre, 二次 for i-- 定义 suf, 三次 for 定义 result, suf*pre 最后 return result 结束战斗

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int len = nums.length;
        int[] pre = new int[len];
        int[] suf = new int[len];
        int[] res = new int[len];

        pre[0] = 1;
        for(int i = 1; i < len; i++){
            pre[i] = pre[i-1] * nums[i-1];
        }

        suf[len-1] = 1;
        for(int i = len-2; i >= 0; i--){
            suf[i] = suf[i+1] * nums[i+1];
        }

        for(int i = 0; i < len; i++){
            res[i] = pre[i] * suf[i];
        }
        return res;
    }
}
```

41. 缺失的第一个正数 【困难】

给你一个未排序的整数数组 `nums`，请你找出其中没有出现的最小的正整数。

请你实现时间复杂度为 $O(n)$ 并且只使用常数级别额外空间的解决方案。

示例 1:

输入: `nums = [1,2,0]`

输出: 3

示例 2:

输入: `nums = [3,4,-1,1]`

输出: 2

💡 **学霸笔记:**

困难直接跳，背了也用不上，用也轮不到我。

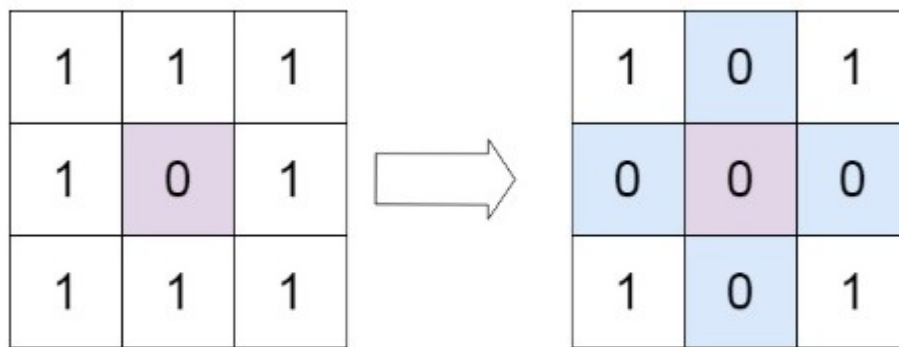
六、矩阵

本类共 4 道题

73. 矩阵置零 【中等】

给定一个 $m \times n$ 的矩阵，如果一个元素为 0，则将其所在行和列的所有元素都设为 0。请使用原地算法。

示例 1:



输入: matrix = [[1,1,1],[1,0,1],[1,1,1]]
输出: [[1,0,1],[0,0,0],[1,0,1]]

📌 学霸笔记:

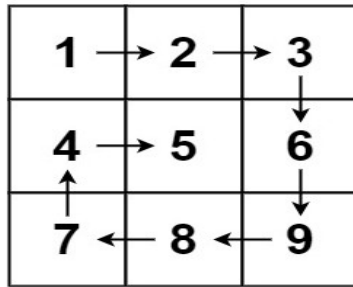
构造横列为 0 和竖列为 0 的数组，第一次两层 for 来遍历原数组赋值给 x, y；第二个两层 for 来判断是不是 x 为 true 或 y 为 true，进行置 0，结束战斗

```
class Solution {
    public void setZeroes(int[][] matrix) {
        int m = matrix.length;
        int n = matrix[0].length;
        boolean[] x = new boolean[m];
        boolean[] y = new boolean[n];
        for(int i = 0; i < m; i++){
            for(int j = 0; j < n; j++){
                if(matrix[i][j] == 0) x[i] = y[j] = true;
            }
        }
        for(int i = 0; i < m; i++){
            for(int j = 0; j < n; j++){
                if(x[i] == true || y[j] == true){
                    matrix[i][j] = 0;
                }
            }
        }
    }
}
```

54. 螺旋矩阵 【中等】

给你一个 m 行 n 列的矩阵 matrix，请按照 顺时针螺旋顺序，返回矩阵中的所有元素。

示例 1:



输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]
输出: [1,2,3,6,9,8,7,4,5]

📌 学霸笔记:

模拟题，想成贪吃蛇，回型遍历四个方向一圈一圈就行。定义上下左右，开 while true 一直循环里面四次 for，第一次左到右，加到 result；for 外判断++上大于下，break，其他三个方向继续…return result 结束战斗

```
class Solution {
    public List<Integer> spiralOrder(int[][] matrix) {
        int s = 0;
        int x = matrix.length - 1;
        int z = 0;
        int y = matrix[0].length - 1;
        List<Integer> result = new ArrayList<>();
        while(true){
            for(int i = z; i <= y; i++) result.add(matrix[s][i]);
            if(++s > x) break;
            for(int i = s; i <= x; i++) result.add(matrix[i][y]);
            if(--y < z) break;
            for(int i = y; i >= z; i--) result.add(matrix[x][i]);
            if(--x < s) break;
            for(int i = x; i >= s; i--) result.add(matrix[i][z]);
            if(++z > y) break;
        }
        return result;
    }
}
```

48. 旋转图像 【中等】

给定一个 $n \times n$ 的二维矩阵 matrix 表示一个图像。请你将图像顺时针旋转 90 度。

你必须在原地旋转图像，这意味着你需要直接修改输入的二维矩阵。请不要使用另一个矩阵来旋转图像。

示例 1:

输入: matrix = [[1,2,3],[4,5,6],[7,8,9]]

输出: [[7,4,1],[8,5,2],[9,6,3]]

示例 2:

输入: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]

输出: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

📌 学霸笔记:

技巧题, 背: 对角线反转一次+单独行反转一次 就行

开两层 for, 注意 j 只走矩形的一半三角形不然重复, 内容就 temp、i->j, j->i 退 for。再开一层 for 丢 matrix[i] 进去 reverse 函数, 函数 reverse 就双指针换一下 ij 就行, 结束战斗

```
class Solution {
    public void rotate(int[][] matrix) {
        int n = matrix.length;
        int m = matrix[0].length;
        for(int i = 0; i < n; i++){
            for(int j = i+1; j < m; j++){
                int temp = matrix[i][j];
                matrix[i][j] = matrix[j][i];
                matrix[j][i] = temp;
            }
        }
        for(int i = 0; i < n; i++){
            reverse(matrix[i]);
        }
    }
    public void reverse(int[] matrix){
        int right = matrix.length - 1;
        int left = 0;
        while(left < right){
            int temp;
            temp = matrix[left];
            matrix[left] = matrix[right];
            matrix[right] = temp;
            left++;
            right--;
        }
    }
}
```

240. 搜索二维矩阵 II 【中等】

编写一个高效的算法来搜索 $m \times n$ 矩阵 `matrix` 中的一个目标值 `target`。该矩阵具有以下特性：

- 每行的元素从左到右升序排列。
- 每列的元素从上到下升序排列。

示例 1：

输入：matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,33]], target = 5

输出：true

💡 学霸笔记：

反方向的钟，从右边开始向左向下找，开 while 循环，还有剩余元素的条件，内容判断都简单，灵神神了。结束战斗

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int i = 0;
        int j = matrix[0].length - 1; // 从右上角开始
        while (i < matrix.length && j >= 0) { // 还有剩余元素
            if (matrix[i][j] == target) {
                return true; // 找到 target
            }
            if (matrix[i][j] < target) {
                i++; // 这一行剩余元素全部小于 target，排除
            } else {
                j--; // 这一列剩余元素全部大于 target，排除
            }
        }
        return false;
    }
}
```

七、链表

本类共 14 道题

160. 相交链表 【简单】

给你两个单链表的头节点 headA 和 headB ，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 null 。

图示两个链表在节点 c1 开始相交：

题目数据 保证 整个链式结构中不存在环。

注意，函数返回结果后，链表必须 保持其原始结构。

示例 1：

输入：intersectVal = 8, listA = [4,1,8,4,5], listB = [5,6,1,8,4,5], skipA = 2, skipB = 3

输出：Intersected at '8'

解释：相交节点的值为 8 （注意，如果两个链表相交则不能为 0）。

示例 2：

输入：intersectVal = 0, listA = [2,6,4], listB = [1,5], skipA = 3, skipB = 2

输出：null

解释：两个链表不相交，因此返回 null。

💡 学霸笔记：

记住 while 遍历，短的遍历完了换头继续走，相遇就是相交点。

定义 a b node 以免头被污染，开 while(ab 不相等)两种情况都考虑了，有香蕉就没毛，没有香蕉都是 null 情况，也退的出来 while ，里面判断是不是 null 是就换头不是就 next；return 结束战斗

```
public class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode a = headA;
        ListNode b = headB;
        while(a != b){
            if(a!=null){
                a=a.next;
            }else{a = headB;}
            if(b!=null){
                b=b.next;
            }else{b = headA;}
        }
        return b;
    }
}
```

206. 反转链表 【简单】

给你单链表的头节点 head ，请你反转链表，并返回反转后的链表。

示例 1:

输入: head = [1,2,3,4,5]

输出: [5,4,3,2,1]

示例 2:

输入: head = [1,2]

输出: [2,1]

💡 学霸笔记:

反转就背：现指向前，前变现，现变后；链表题 return 时候要看 curr 有没有走过，走过了就用 prev。

```

class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;    // 已反转部分的头
        ListNode curr = head;    // 当前要处理的节点

        while (curr != null) {
            ListNode next = curr.next; // 1. 先保存下一个（不然断了找不到）
            curr.next = prev;          // 2. 当前指向前一个（反转核心！）
            prev = curr;               // 3. prev 前移
            curr = next;               // 4. curr 前移
        }
        return prev;                // 最后 prev 就是新头
    }
}

```

234. 回文链表 【简单】

给你一个单链表的头节点 `head`，请你判断该链表是否为回文链表。如果是，返回 `true`；否则，返回 `false`。

示例 1：

输入：head = [1,2,2,1]

输出：true

示例 2：

输入：head = [1,2]

输出：false

💡 学霸笔记：

我的简单思路是转 list 后用双指针判断 `==`，做的好回去等通知吧。优化思路是快慢指针跑完就是中间点和终点，从中间点开始反转后半链表，再做比较判断就行。评价为没必要记住茴四种写法

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        List<Integer> list = new ArrayList<>();
        ListNode node = head;
        while(node != null){
            list.add(node.val);
            node = node.next;
        }
        int left = 0;
        int right = list.size() - 1;
        while(left < right){
            if(list.get(left) != list.get(right)) return false;
            left++;
            right--;
        }
        return true;
    }
}
```

141. 环形链表 【简单】

给你一个链表的头节点 head ，判断链表中是否有环。

如果链表中有某个节点，可以通过连续跟踪 next 指针再次到达，则链表中存在环。 为了表示给定链表中的环，评测系统内部使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。注意：pos 不作为参数进行传递。仅仅是为了标识链表的实际情况。

如果链表中存在环 ，则返回 true 。 否则，返回 false 。

示例 1：

输入：head = [3,2,0,-4], pos = 1

输出：true

解释：链表中有一个环，其尾部连接到第二个节点。

示例 2：

输入: head = [1,2], pos = 0

输出: true

📖 学霸笔记:

bro u got it,fast and slow pointer method can do that,u no what I mean,you will make sure it that null scene,that is noway to access the algo,end fighting

```
public class Solution {
    public boolean hasCycle(ListNode head) {
        if(head == null || head.next == null) return false;
        ListNode fast = head.next;
        ListNode slow = head;
        while(fast != slow){
            if(fast.next != null && fast.next.next != null) fast = fast.next.next; else{return false;}
            if(slow.next != null) slow = slow.next;else{return false;}
        }
        if(slow == null) return false;
        return true;
    }
}
```

142. 环形链表 II 【中等】

给定一个链表的头节点 head，返回链表开始入环的第一个节点。如果链表无环，则返回 null。

如果链表中有某个节点，可以通过连续跟踪 next 指针再次到达，则链表中存在环。为了表示给定链表中的环，评测系统内部使用整数 pos 来表示链表尾连接到链表中的位置（索引从 0 开始）。如果 pos 是 -1，则在该链表中没有环。注意：pos 不作为参数进行传递，仅仅是为了标识链表的实际情况。

不允许修改 链表。

示例 1:

输入: head = [3,2,0,-4], pos = 1

输出：返回索引为 1 的链表节点

解释：链表中有一个环，其尾部连接到第二个节点。

示例 2：

输入：head = [1,2], pos = 0

输出：返回索引为 0 的链表节点

📖 学霸笔记：

依旧背：快慢指针 while 走一遍，走不完找到=就有环，走完了就没环 null。第二个 while 走一样速度，走到=就代表切环点，原理看数学证明，直接背就完了结束战斗

```
public class Solution {
    public ListNode detectCycle(ListNode head) {
        ListNode fast = head;
        ListNode slow = head;
        if(head == null || head.next == null) return null;
        while(true){
            if(fast.next != null && fast.next.next != null){
                fast = fast.next.next;
            }else{return null;}
            if(slow.next != null){
                slow = slow.next;
            }else{return null;}
            if(slow == fast) break;
        }
        fast = head;
        while(fast != slow){
            if(fast.next != null) fast = fast.next;
            if(slow.next != null) slow = slow.next;
        }
        return fast;
    }
}
```

21. 合并两个有序链表 【简单】

将两个升序链表合并为一个新的 升序 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

示例 1:

输入: l1 = [1,2,4], l2 = [1,3,4]

输出: [1,1,2,3,4,4]

示例 2:

输入: l1 = [], l2 = []

输出: []

📌 学霸笔记:

用禁忌之力 Collections 的 sort 排序一下，在重新构造一个链表 return 就行。注意 int[] 是 Arrays 的 sort，list 是 Collections 的，result 的虚拟头，return next 才指真头，结束战斗

```
class Solution {
    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        List<Integer> temp = new ArrayList<>();
        ListNode node1 = list1;
        ListNode node2 = list2;
        while(node1 != null){
            temp.add(node1.val);
            node1 = node1.next;
        }
        while(node2 != null){
            temp.add(node2.val);
            node2 = node2.next;
        }
        Collections.sort(temp);
        ListNode result = new ListNode(0);
        ListNode curr = result;
        for(int i : temp){
            curr.next = new ListNode(i);
            curr = curr.next;
        }
        return result.next;
    }
}
```

2. 两数相加 【中等】

给你两个 非空 的链表，表示两个非负的整数。它们每位数字都是按照 逆序 的方式存储的，并且每个节点只能存储 一位 数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。

你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

示例 1：

输入： l1 = [2,4,3], l2 = [5,6,4]

输出： [7,0,8]

解释： 342 + 465 = 807

示例 2：

输入： l1 = [9,9,9,9], l2 = [9,9,9]

输出： [8,9,9,0,1]

💡 **学霸笔记：**

逻辑不直接，直接开背吧。定义虚拟头和进位 carry，开 while 条件是两个链表还有或者进位还有一个成立就行，里面算 sum(别忘了 val 可能 null 要？判断下) carry 用/，next 指向构造的 node,值就是%10，自己 curr 前进，链表 l1 l2 前进，return dummy.next 结束战斗

```

class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        int carry = 0;
        while(l1 != null || l2 != null || carry != 0){
            int a = l1 != null ? l1.val : 0;
            int b = l2 != null ? l2.val : 0;
            int sum = a + b + carry;
            carry = sum / 10;
            curr.next = new ListNode(sum % 10);
            curr = curr.next;
            if(l1 != null) l1 = l1.next;
            if(l2 != null) l2 = l2.next;
        }
        return dummy.next;
    }
}

```

19. 删除链表的倒数第 N 个结点 【中等】

给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

示例 1:

输入: head = [1,2,3,4,5], n = 2

输出: [1,2,3,5]

示例 2:

输入: head = [1], n = 1

输出: []

📌 学霸笔记:

快指针先跑 n 个，慢指针后跑，到快指针链表完结时候就正好慢指针指到要删，因为链表不好倒退，就可以前面快指针多跑一个，让 $next = next.next$ 后 return 虚拟头 next 结束战斗

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode prev = dummy;
        ListNode curr = dummy;
        int m = n;
        while(m>=0){
            curr = curr.next;
            m--;
        }
        while(curr != null){
            curr = curr.next;
            prev = prev.next;
        }
        prev.next = prev.next.next;
        return dummy.next;
    }
}
```

24. 两两交换链表中的节点 【中等】

给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在`不修改节点内部`的值的条件下完成本题（即，只能进行节点交换）。

示例 1：

输入：head = [1,2,3,4]

输出：[2,1,4,3]

示例 2：

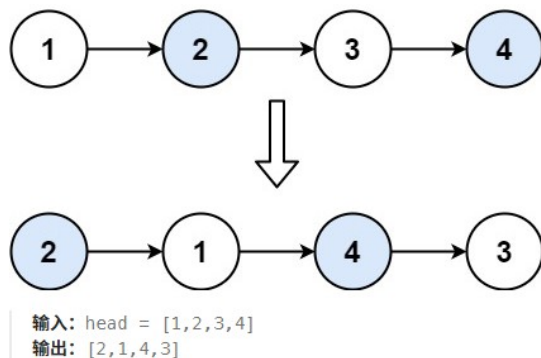
输入：head = []

输出：[]

📌 学霸笔记：

用虚拟头指 head，while 至少两个节点开始，定义 node23 方便指，0 指 2，2 指 1，1 指 3，指针变完了该节点前进，0 变 1，1 变 3，最后 return 虚拟头 next 结束战斗。

示例 1:



```
30
31 class Solution {
32     public ListNode swapPairs(ListNode head) {
33         ListNode dummy = new ListNode(0, head);
34         ListNode node0 = dummy;
35         ListNode node1 = head;
36
37         while (node1 != null && node1.next != null) {
38             ListNode node2 = node1.next;
39             ListNode node3 = node2.next;
40
41             node0.next = node2;
42             node2.next = node1;
43             node1.next = node3;
44             node0 = node1;
45             node1 = node3;
46         }
47         return dummy.next;
48     }
}
```

25. K 个一组翻转链表 【困难】

给你链表的头节点 head，每 k 个节点一组进行翻转，请你返回修改后的链表。

k 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 k 的整数倍，那么请将最后剩余的节点保持原有顺序。

你不能只是单纯的改变节点内部的值，而是需要实际进行节点交换。

示例 1:

输入: head = [1,2,3,4,5], k = 2

输出: [2,1,4,3,5]

示例 2:

输入: head = [1,2,3,4,5], k = 3

输出: [3,2,1,4,5]

📌 学霸笔记:

两个凡是，凡是考难题的就是为难你；凡是不考你 hot100 的就是不认真对待你

138. 随机链表的复制 【中等】

给你一个长度为 n 的链表，每个节点包含一个额外增加的随机指针 `random`，该指针可以指向链表中的任何节点或空节点。

构造这个链表的深拷贝。深拷贝应该正好由 n 个全新节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。复制链表中的指针都不应指向原链表中的节点。

返回复制链表的头节点。

示例 1：

输入：`head = [[7,null],[13,0],[11,4],[10,2],[1,0]]`

输出：`[[7,null],[13,0],[11,4],[10,2],[1,0]]`

示例 2：

输入：`head = [[1,1],[2,1]]`

输出：`[[1,1],[2,1]]`

💡 学霸笔记：

看了好像优先级不高，没人面，看遍思路过了。

要复制一遍很简单，但是 `random` 指向可能在后面还没构造完，想到哈希表法，但是这个要遍历两次(而且比这个难背)，用在原链表插入一个原链表 `ab->aa`bb`...`，这样 `random` 就好插了，后面再分离一波结束战斗

又看了一眼好像哈希表法更好背，我有点傻逼了，for 第一遍单纯 map 赋值让 map 存 key 原 value 新 不涉及 `next` 和 `random`，for 第二遍赋值 `next` 和 `random` 最后 return 结束战斗

```

public Node copyRandomList(Node head) {
    //1构造交错链表
    //2遍历交错链表，把原 random 指向的节点也复制一份，规则是 新的 random 指向是原 random 指向的下一个
    //3分离交错链表，return 答案
    for(Node node = head;node != null;node = node.next.next){
        node.next = new Node(node.val,node.next);
    }

    for(Node node = head;node != null;node = node.next.next){
        if(node.random != null)
            node.next.random = node.random.next;
    }
    Node dummy = new Node(0);
    Node tail = dummy;
    for(Node node = head;node != null;node = node.next,tail=tail.next){
        tail.next = node.next;
        node.next = node.next.next;
    }
    return dummy.next;
}

class Solution{
public Node copyRandomList(Node head) {
    Map<Node, Node> map = new HashMap<>();

    // 第一遍：先把所有新节点造出来，建立原节点 → 新节点的映射
    for (Node node = head; node != null; node = node.next) {
        map.put(node, new Node(node.val));
    }

    // 第二遍：把所有指针关系接上
    for (Node node = head; node != null; node = node.next) {
        map.get(node).next = map.get(node.next);
        map.get(node).random = map.get(node.random);
    }

    return map.get(head);
}
}

```

148. 排序链表 【中等】

给你链表的头结点 head ，请将其按升序 排列并返回 排序后的链表。

进阶：

- 你可以在 $O(n \log n)$ 时间复杂度和常数级空间复杂度下，对链表进行排序吗？

示例 1：

输入：head = [4,2,1,3]

输出：[1,2,3,4]

示例 2：

输入：head = [-1,5,3,4,0]

输出：[-1,0,3,4,5]

📌 学霸笔记：

一遍过，用禁忌之力就是爽。

```
class Solution {
    public ListNode sortList(ListNode head) {
        if(head == null) return head;
        List<Integer> list = new ArrayList<>();
        ListNode node = head;
        while(node != null){
            list.add(node.val);
            node = node.next;
        }
        Collections.sort(list);
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        for(int i : list){
            curr.next = new ListNode(i);
            curr = curr.next;
        }
        return dummy.next;
    }
}
```

23. 合并 K 个升序链表 【困难】

给你一个链表数组，每个链表都已经按升序排列。

请你将所有链表合并到一个升序链表中，返回合并后的链表。

示例 1:

输入: lists = [[1,4,5],[1,3,4],[2,6]]

输出: [1,1,2,3,4,4,5,6]

示例 2:

输入: lists = []

输出: []

📌 学霸笔记:

我合并格调，不做。

狗不做我做，这题直接用禁忌之力 list 的 sort 解决。别管什么优先队列分治合并了。吃我一击吧。

```
class Solution {
    public ListNode mergeKLists(ListNode[] lists) {
        List<Integer> list = new ArrayList<>();
        for (ListNode head : lists) {
            ListNode node = head;
            while (node != null) {
                list.add(node.val);
                node = node.next;
            }
        }
        Collections.sort(list);
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        for (int val : list) {
            curr.next = new ListNode(val);
            curr = curr.next;
        }
        return dummy.next;
    }
}
```

146. LRU 缓存 【中等】

请你设计并实现一个满足 LRU (最近最少使用) 缓存约束的数据结构。

实现 LRU Cache 类：

- LRU Cache(int capacity) 以正整数作为容量 capacity 初始化 LRU 缓存。
- int get(int key) 如果关键字 key 存在于缓存中，则返回关键字的值，否则返回 -1。
- void put(int key, int value) 如果关键字 key 已经存在，则变更其数据值 value；如果不存在，则向缓存中插入该组 key-value。如果插入操作导致关键字数量超过 capacity，则应该逐出最久未使用的关键字。

函数 get 和 put 必须以 $O(1)$ 的平均时间复杂度运行。

示例：

输入：

["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]

[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]

输出: [null, null, null, 1, null, -1, null, -1, 3, 4]

📌 学霸笔记：

大厂赛级吟唱背诵题，凡人看看就行了。不是人人都叫韩立，我们可以是路明非，没有小恶魔…

```
class LRUCache {
    class DLinkedNode {
        int key, value;
        DLinkedNode prev, next;
        DLinkedNode() {}
        DLinkedNode(int key, int value) { this.key = key; this.value = value; }
    }
    private Map<Integer, DLinkedNode> cache = new HashMap<>();
    private int capacity;
    private DLinkedNode head, tail; // 两个哨兵
    public LRUCache(int capacity) {
        this.capacity = capacity;
        head = new DLinkedNode();
        tail = new DLinkedNode();
        head.next = tail;
        tail.prev = head;
    }
    public int get(int key) {
        DLinkedNode node = cache.get(key);
        if (node == null) return -1;
        moveToHead(node);
        return node.value;
    }
    public void put(int key, int value) {
        DLinkedNode node = cache.get(key);
        if (node == null) {
            DLinkedNode newNode = new DLinkedNode(key, value);
            cache.put(key, newNode);
            addToHead(newNode);
        }
    }
}
```

```

        addToHead(newNode);
        if (cache.size() > capacity) {
            DLinkedNode removed = removeTail();
            cache.remove(removed.key); // ← 关键：map 也要删
        }
    } else {
        node.value = value;
        moveToHead(node);
    }
}

// ===== 四个工具方法 =====
private void addToHead(DLinkedNode node) {
    node.prev = head;
    node.next = head.next;
    head.next.prev = node;
    head.next = node;
}

private void removeNode(DLinkedNode node) {
    node.prev.next = node.next;
    node.next.prev = node.prev;
}

private void moveToHead(DLinkedNode node) {
    removeNode(node);
    addToHead(node);
}

private DLinkedNode removeTail() {
    DLinkedNode node = tail.prev; // tail 前面那个就是最久没用的
    removeNode(node);
    return node;
}

```

八、二叉树

本类共 15 道题

94. 二叉树的中序遍历 【简单】

给定一个二叉树的根节点 root，返回它的 中序 遍历。

示例 1:

输入: root = [1,null,2,3]

输出: [1,3,2]

示例 2:

输入: root = []

输出: []

💡 学霸笔记:

104. 二叉树的最大深度 【简单】

给定一个二叉树 root，返回其最大深度。

二叉树的 最大深度 是指从根节点到最远叶子节点的最长路径上的节点数。

示例 1:

输入: root = [3,9,20,null,null,15,7]

输出: 3

示例 2:

输入: root = [1,null,2]

输出: 2

💡 学霸笔记:

226. 翻转二叉树 【简单】

给你一棵二叉树的根节点 root，翻转这棵二叉树，并返回其根节点。

示例 1:

输入: **root** = [4,2,7,1,3,6,9]

输出: [4,7,2,9,6,3,1]

示例 2:

输入: **root** = [2,1,3]

输出: [2,3,1]

📖 学霸笔记:

101. 对称二叉树 【简单】

给你一个二叉树的根节点 **root** , 检查它是否轴对称。

示例 1 :

输入 : **root** = [1,2,2,3,4,4,3]

输出 : **true**

示例 2 :

输入 : **root** = [1,2,2,null,3,null,3]

输出 : **false**

📖 学霸笔记:

543. 二叉树的直径 【简单】

给你一棵二叉树的根节点，返回该树的直径。

二叉树的直径是指树中任意两个节点之间最长路径的长度。这条路径可能经过也可能不经过根节点 root。

两节点之间路径的长度由它们之间边数表示。

示例 1:

输入: root = [1,2,3,4,5]

输出: 3

解释: 3，取路径 [4,2,1,3] 或 [5,2,1,3] 的长度。

示例 2:

输入: root = [1,2]

输出: 1

💡 学霸笔记:

102. 二叉树的层序遍历 【中等】

给你二叉树的根节点 root，返回其节点值的层序遍历。（即逐层地，从左到右访问所有节点）。

示例 1:

输入: root = [3,9,20,null,null,15,7]

输出: [[3],[9,20],[15,7]]

示例 2:

输入: `root = [1]`

输出: `[[1]]`

💡 学霸笔记:

108. 将有序数组转换为二叉搜索树 【简单】

给你一个整数数组 `nums`，其中元素已经按升序排列，请你将其转换为一棵高度平衡二叉搜索树。

高度平衡二叉树是一棵满足「每个节点的左右两个子树的高度差的绝对值不超过 1」的二叉树。

示例 1:

输入: `nums = [-10,-3,0,5,9]`

输出: `[0,-3,9,-10,null,5]`

示例 2:

输入: `nums = [1,3]`

输出: `[3,1]`

💡 学霸笔记:

98. 验证二叉搜索树 【中等】

给你一个二叉树的根节点 `root`，判断其是否是一个有效的二叉搜索树。

有效 二叉搜索树定义如下：

- 节点的左子树只包含 小于 当前节点的数。
- 节点的右子树只包含 大于 当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

示例 1：

输入：**root = [2,1,3]**

输出：**true**

示例 2：

输入：**root = [5,1,4,null,null,3,6]**

输出：**false**

解释：根节点的值是 **5**，但是右子节点的值是 **4**。

📌 学霸笔记：

230. 二叉搜索树中第 K 小的元素 【中等】

给定一个二叉搜索树的根节点 root，和一个整数 k，请你设计一个算法查找其中第 k 小的元素（从 1 开始计数）。

示例 1：

输入：**root = [3,1,4,null,2], k = 1**

输出：**1**

示例 2：

输入：**root = [5,3,6,2,4,null,null,1], k = 3**

输出：**3**

📌 学霸笔记：

199. 二叉树的右视图 【中等】

给定一个二叉树的根节点 `root`，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

示例 1：

输入：`root = [1,2,3,null,5,null,4]`

输出：`[1,3,4]`

示例 2：

输入：`root = [1,null,3]`

输出：`[1,3]`

💡 学霸笔记：

114. 二叉树展开为链表 【中等】

给你二叉树的根结点 `root`，请你将它展开为一个单链表：

- 展开后的单链表应该同样使用 `TreeNode`，其中 `right` 子指针指向链表中下一个结点，而左子指针始终为 `null`。

- 展开后的单链表应该与二叉树 先序遍历 顺序相同。

示例 1：

输入：`root = [1,2,5,3,4,null,6]`

输出：`[1,null,2,null,3,null,4,null,5,null,6]`

示例 2：

输入：`root = []`

输出：`[]`

💡 学霸笔记：

105. 从前序与中序遍历序列构造二叉树 【中等】

给定两个整数数组 `preorder` 和 `inorder`，其中 `preorder` 是二叉树的先序遍历，`inorder` 是同一棵树的中序遍历，请构造二叉树并返回其根节点。

示例 1：

输入：`preorder = [3,9,20,15,7]`, `inorder = [9,3,15,20,7]`

输出：`[3,9,20,null,null,15,7]`

示例 2：

输入：`preorder = [-1]`, `inorder = [-1]`

输出：`[-1]`

💡 学霸笔记：

437. 路径总和 III 【中等】

给定一个二叉树的根节点 `root`，和一个整数 `targetSum`，求该二叉树里节点值之和等于 `targetSum` 的路径的数目。

路径不需要从根节点开始，也不需要叶子节点结束，但是路径方向必须是向下的（只能从父节点到子节点）。

示例 1：

输入：**root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8**

输出：**3**

解释：和等于 8 的路径有 3 条，如图所示。

示例 2：

输入：**root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22**

输出：**3**

💡 学霸笔记：

236. 二叉树的最近公共祖先 【中等】

给定一个二叉树，找到该树中两个指定节点的最近公共祖先。

百度百科中最近公共祖先的定义为："对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）"。

示例 1：

输入：**root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1**

输出：**3**

解释：节点 5 和节点 1 的最近公共祖先是节点 3。

示例 2：

输入：**root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4**

输出：**5**

解释：节点 5 和节点 4 的最近公共祖先是节点 5。

💡 学霸笔记：

124. 二叉树中的最大路径和 【困难】

二叉树中的 路径 被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 至多出现一次 。该路径 至少包含一个 节点，且不一定经过根节点。

路径和 是路径中各节点值的总和。

给你一个二叉树的根节点 `root` ，返回其 最大路径和 。

示例 1：

输入：`root = [1,2,3]`

输出：`6`

解释：最优路径是 `2 -> 1 -> 3` ，路径和为 $2 + 1 + 3 = 6$

示例 2：

输入：`root = [-10,9,20,null,null,15,7]`

输出：`42`

解释：最优路径是 `15 -> 20 -> 7` ，路径和为 $15 + 20 + 7 = 42$

💡 **学霸笔记：**

两个凡是，凡是考难题的就是为难你；凡是不考你 hot100 的就是不认真对待你

九、图论

本类共 4 道题

200. 岛屿数量 【中等】

给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

示例 1：

输入： `grid = [`

```
["1","1","1","1","0"],
["1","1","0","1","0"],
["1","1","0","0","0"],
["0","0","0","0","0"]
]
```

输出： `1`

示例 2：

输入： `grid = [`

```
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
```

输出： `3`

📌 **学霸笔记：**

994. 腐烂的橘子 【中等】

在给定的 $m \times n$ 网格 `grid` 中，每个单元格可以有以下三个值之一：

- 值 0 代表空单元格；
- 值 1 代表新鲜橘子；
- 值 2 代表腐烂的橘子。每分钟，腐烂的橘子 周围 4 个方向上相邻 的新鲜橘子都会腐烂。

返回 直到单元格中没有新鲜橘子为止所必须经过的最小分钟数。如果不可能，返回 -1 。

示例 1：

输入： `grid = [[2,1,1],[1,1,0],[0,1,1]]`

输出： 4

示例 2：

输入： `grid = [[2,1,1],[0,1,1],[1,0,1]]`

输出： -1

解释： 左下角的橘子（第 2 行， 第 0 列）永远不会腐烂，因为腐烂只会发生在 4 个正上方。

📖 学霸笔记：

207. 课程表 【中等】

你这个学期必须选修 `numCourses` 门课程，记为 0 到 `numCourses - 1` 。

在选修某些课程之前需要一些先修课程。 先修课程按数组 `prerequisites` 给出，其中 `prerequisites[i] = [ai, bi]` ，表示如果要学习课程 `ai` 则 必须 先学习课程 `bi` 。

例如，先修课程对 [0, 1] 表示：想要学习课程 0，你需要先完成课程 1。

请你判断是否可能完成所有课程的学习？如果可以，返回 true；否则，返回 false。

示例 1：

输入：**numCourses = 2, prerequisites = [[1,0]]**

输出：**true**

解释：总共有 2 门课程。学习课程 1 之前，你需要完成课程 0。这是可能的。

示例 2：

输入：**numCourses = 2, prerequisites = [[1,0],[0,1]]**

输出：**false**

解释：总共有 2 门课程。学习课程 1 之前，你需要先完成课程 0；并且学习课程 0 之前，你还应先完成课程 1。这是不可能的。

💡 学霸笔记：

208. 实现 Trie (前缀树) 【中等】

Trie（发音类似 "try"）或者说 前缀树 是一种树形数据结构，用于高效地存储和检索字符串数据集中的键。这一数据结构有相当多的应用情景，例如自动补完和拼写检查。

请你实现 Trie 类：

- Trie() 初始化前缀树对象。
- void insert(String word) 向前缀树中插入字符串 word。
- boolean search(String word) 如果字符串 word 在前缀树中，返回 true（即，在检索之前已经插入）；否则，返回 false。

- boolean startsWith(String prefix) 如果之前已经插入的字符串 word 的前缀之一为 prefix ，返回 true ；否则，返回 false 。

示例：

输入：

["Trie", "insert", "search", "search", "startsWith", "insert", "search"]

[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]

输出：**[null, null, true, false, true, null, true]**

💡 学霸笔记：

十、回溯

本类共 8 道题

46. 全排列 【中等】

给定一个不含重复数字的数组 nums ，返回其 所有可能的全排列。你可以 按任意顺序 返回答案。

示例 1：

输入：nums = [1,2,3]

输出：[[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

示例 2：

输入：nums = [0,1]

输出：[[0,1],[1,0]]

💡 学霸笔记：

78. 子集 【中等】

给你一个整数数组 `nums`，数组中的元素互不相同。返回该数组所有可能的子集（幂集）。

解集不能包含重复的子集。你可以按任意顺序返回解集。

示例 1：

输入：`nums = [1,2,3]`

输出：`[[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

示例 2：

输入：`nums = [0]`

输出：`[[],[0]]`

📖 学霸笔记：

17. 电话号码的字母组合 【中等】

给定一个仅包含数字 2-9 的字符串，返回所有它能表示的字母组合。答案可以按任意顺序返回。

给出数字到字母的映射如下（与电话按键相同）。注意 1 不对应任何字母。

示例 1：

输入：`digits = "23"`

输出：`["ad","ae","af","bd","be","bf","cd","ce","cf"]`

示例 2：

输入：`digits = ""`

输出：`[]`

📌 学霸笔记：

39. 组合总和 【中等】

给你一个 无重复元素 的整数数组 `candidates` 和一个目标整数 `target` ，找出 `candidates` 中可以使数字和为目标数 `target` 的 所有 不同组合 ，并以列表形式返回。你可以按 任意顺序 返回这些组合。

`candidates` 中的 同一个 数字可以 无限制重复被选取 。如果至少一个数字的被选数量不同，则两种组合是不同的。

对于给定的输入，保证和为 `target` 的不同组合数少于 150 个。

示例 1：

输入：`candidates = [2,3,6,7], target = 7`

输出：`[[2,2,3],[7]]`

示例 2：

输入：`candidates = [2,3,5], target = 8`

输出：`[[2,2,2,2],[2,3,3],[3,5]]`

📌 学霸笔记：

22. 括号生成 【中等】

数字 n 代表生成括号的对数，请你设计一个函数，用于能够生成所有可能的并且有效的括号组合。

示例 1：

输入： $n = 3$

输出：["((())", "(()())", "(())()", "()()()", "()(())"]

示例 2：

输入： $n = 1$

输出：["()"]

💡 学霸笔记：

79. 单词搜索 【中等】

给定一个 $m \times n$ 二维字符网格 `board` 和一个字符串单词 `word`。如果 `word` 存在于网格中，返回 `true`；否则，返回 `false`。

单词必须按照字母顺序，通过相邻的单元格内的字母构成，其中“相邻”单元格是那些水平相邻或垂直相邻的单元格。同一个单元格内的字母不允许被重复使用。

示例 1：

输入：`board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "ABCCED"`

输出：`true`

示例 2：

输入：`board = [ ["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"] ]`, `word = "SEE"`

输出：`true`

📖 学霸笔记:

131. 分割回文串 【中等】

给你一个字符串 s ，请你将 s 分割成一些子串，使每个子串都是回文串。返回 s 所有可能的分割方案。

示例 1：

输入： $s = \text{"aab"}$

输出： $[[\text{"a"}, \text{"a"}, \text{"b"}], [\text{"aa"}, \text{"b"}]]$

示例 2：

输入： $s = \text{"a"}$

输出： $[[\text{"a"}]]$

📖 学霸笔记:

51. N 皇后 【困难】

按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。

n 皇后问题 研究的是如何将 n 个皇后放置在 $n \times n$ 的棋盘上，并且使皇后彼此之间不能相互攻击。

给你一个整数 n ，返回所有不同的 n 皇后问题的解决方案。

每一种解法包含一个不同的 n 皇后问题的棋子放置方案，该方案中 'Q' 和 '.' 分别代表了皇后和空位。

示例 1:

输入: $n = 4$

输出: `[[".Q..", "...Q", "Q...", "..Q."], ["..Q.", "Q...", "...Q", ".Q.."]]`

示例 2:

输入: $n = 1$

输出: `[["Q"]]`

💡 学霸笔记:

不写，困难写格调。考你困难就是不想让你过。

十一、二分查找

本类共 6 道题

35. 搜索插入位置 【简单】

给定一个排序数组和一个目标值，在数组中找到目标值，并返回其索引。如果目标值不存在于数组中，返回它将会被按顺序插入的位置。

请务必使用时间复杂度为 $O(\log n)$ 的算法。

示例 1:

输入: `nums = [1,3,5,6], target = 5`

输出: `2`

示例 2:

输入: `nums = [1,3,5,6], target = 2`

输出: `1`

💡 学霸笔记：

74. 搜索二维矩阵 【中等】

给你一个满足下述两条属性的 $m \times n$ 整数矩阵：

- 每行中的整数从左到右按非严格递增顺序排列。
- 每行的第一个整数大于前一行的最后一个整数。

给你一个整数 `target`，如果 `target` 在矩阵中，返回 `true`；否则，返回 `false`。

示例 1：

输入：`matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`

输出：`true`

示例 2：

输入：`matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 13`

输出：`false`

💡 学霸笔记：

34. 在排序数组中查找元素的第一个和最后一个位置 【中等】

给你一个按照非递减顺序排列的整数数组 `nums`，和一个目标值 `target`。请你找出给定目标值在数组中的开始位置和结束位置。

如果数组中不存在目标值 `target`，返回 `[-1, -1]`。

你必须设计并实现时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1：

输入：`nums = [5,7,7,8,8,10]`, `target = 8`

输出：`[3,4]`

示例 2：

输入：`nums = [5,7,7,8,8,10]`, `target = 6`

输出：`[-1,-1]`

💡 学霸笔记：

33. 搜索旋转排序数组 【中等】

整数数组 `nums` 按升序排列，数组中的值 互不相同。

在传递给函数之前，`nums` 在预先未知的某个下标 k ($0 \leq k < \text{nums.length}$) 上进行了旋转，使数组变为 `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]`（下标从 0 开始计数）。例如，`[0,1,2,4,5,6,7]` 在下标 3 处经旋转后可能变为 `[4,5,6,7,0,1,2]`。

给你旋转后的数组 `nums` 和一个整数 `target`，如果 `nums` 中存在这个目标值 `target`，则返回它的下标，否则返回 `-1`。

你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1：

输入：**nums = [4,5,6,7,0,1,2], target = 0**

输出：**4**

示例 2：

输入：**nums = [4,5,6,7,0,1,2], target = 3**

输出：**-1**

📖 学霸笔记：

153. 寻找旋转排序数组中的最小值 【中等】

已知一个长度为 n 的数组，预先按照升序排列，经由 1 到 n 次旋转后，得到输入数组。例如，原数组 $\text{nums} = [0,1,2,4,5,6,7]$ 在变化后可能得到：

- 若旋转 4 次，则可以得到 $[4,5,6,7,0,1,2]$
- 若旋转 7 次，则可以得到 $[0,1,2,4,5,6,7]$

注意，数组 $[a[0], a[1], a[2], \dots, a[n-1]]$ 旋转一次的结果为数组 $[a[n-1], a[0], a[1], a[2], \dots, a[n-2]]$ 。

给你一个元素值互不相同的数组 nums ，它原来是一个升序排列的数组，并按上述情形进行了多次旋转。请你找出并返回数组中的最小元素。

你必须设计一个时间复杂度为 $O(\log n)$ 的算法解决此问题。

示例 1：

输入：**nums = [3,4,5,1,2]**

输出：**1**

示例 2：

输入: `nums = [4,5,6,7,0,1,2]`

输出: `0`

💡 学霸笔记:

4. 寻找两个正序数组的中位数 【困难】

给定两个大小分别为 m 和 n 的正序（从小到大）数组 `nums1` 和 `nums2`。请你找出并返回这两个正序数组的中位数。

算法的时间复杂度应该为 $O(\log(m+n))$ 。

示例 1:

输入: `nums1 = [1,3]`, `nums2 = [2]`

输出: `2.00000`

解释: 合并数组 = `[1,2,3]`，中位数 `2`

示例 2:

输入: `nums1 = [1,2]`, `nums2 = [3,4]`

输出: `2.50000`

解释: 合并数组 = `[1,2,3,4]`，中位数 $(2 + 3) / 2 = 2.5$

💡 学霸笔记:

两个凡是，凡是考困难题的就是为难你；凡是不考你 hot100 的就是不认真对待你

十二、栈

本类共 5 道题

20. 有效的括号 【简单】

给定一个只包括 '(' , ')' , '{' , '}' , '[' , ']' 的字符串 s ，判断字符串是否有效。

有效字符串需满足：

1. 左括号必须用相同类型的右括号闭合。
2. 左括号必须以正确的顺序闭合。
3. 每个右括号都有一个对应的相同类型的左括号。

示例 1：

输入：s = "()"

输出：true

示例 2：

输入：s = "()[]{}"

输出：true

📖 学霸笔记：

155. 最小栈 【中等】

设计一个支持 push ， pop ， top 操作，并能在常数时间内检索到最小元素的栈。

实现 MinStack 类：

- MinStack() 初始化堆栈对象。

- void push(int val) 将元素 val 推入堆栈。
- void pop() 删除堆栈顶部的元素。
- int top() 获取堆栈顶部的元素。
- int getMin() 获取堆栈中的最小元素。

示例 1：

输入：

```
["MinStack","push","push","push","getMin","pop","top","getMin"]
```

```
[[],[-2],[0],[-3],[],[],[],[ ]]
```

输出：**[null,null,null,null,-3,null,0,-2]**

解释：

```
MinStack minStack = new MinStack();
```

```
minStack.push(-2);
```

```
minStack.push(0);
```

```
minStack.push(-3);
```

```
minStack.getMin(); --> 返回 -3.
```

```
minStack.pop();
```

```
minStack.top(); --> 返回 0.
```

```
minStack.getMin(); --> 返回 -2.
```

📌 学霸笔记：

394. 字符串解码 【中等】

给定一个经过编码的字符串，返回它解码后的字符串。

编码规则为: k[encoded_string]，表示其中方括号内部的 encoded_string 正好重复 k 次。注意 k 保证为正整数。

你可以认为输入字符串总是有效的；输入字符串中没有额外的空格，且输入的方括号总是符合格式要求的。

此外，你可以认为原始数据不包含数字，所有的数字只表示重复的次数 k ，例如不会出现像 $3a$ 或 $2[4]$ 的输入。

示例 1：

输入： $s = "3[a]2[bc]"$

输出： $"aaabcbc"$

示例 2：

输入： $s = "3[a2[c]]"$

输出： $"accaccacc"$

💡 学霸笔记：

739. 每日温度 【中等】

给定一个整数数组 `temperatures`，表示每天的温度，返回一个数组 `answer`，其中 `answer[i]` 是指对于第 i 天，下一个更高温度出现在几天后。如果气温在这之后都不会升高，请在该位置用 `0` 来代替。

示例 1：

输入：`temperatures = [73,74,75,71,69,72,76,73]`

输出：`[1,1,4,2,1,1,0,0]`

示例 2：

输入：`temperatures = [30,40,50,60]`

输出：`[1,1,1,0]`

💡 学霸笔记：

84. 柱状图中最大的矩形 【困难】

给定 n 个非负整数，用来表示柱状图中各个柱子的高度。每个柱子彼此相邻，且宽度为 1。

求在该柱状图中，能够勾勒出来的矩形的最大面积。

示例 1:

输入: `heights = [2,1,5,6,2,3]`

输出: 10

解释: 最大的矩形为图中红色区域，面积为 10

示例 2:

输入: `heights = [2,4]`

输出: 4

💡 学霸笔记:

两个凡是，凡是考困难的就是为难你；凡是不考你 hot100 的就是不认真对待你

十三、堆

本类共 3 道题

215. 数组中的第 K 个最大元素 【中等】

给定整数数组 `nums` 和整数 k ，请返回数组中第 k 个最大的元素。

请注意，你需要找的是数组排序后的第 k 个最大的元素，而不是第 k 个不同的元素。

你必须设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

示例 1:

输入: [3,2,1,5,6,4], k = 2

输出: 5

示例 2:

输入: [3,2,3,1,2,4,5,5,6], k = 4

输出: 4

💡 **学霸笔记:**

347. 前 K 个高频元素 【中等】

给你一个整数数组 `nums` 和一个整数 `k`，请你返回其中出现频率前 `k` 高的元素。你可以按任意顺序返回答案。

示例 1:

输入: `nums = [1,1,1,2,2,3]`, k = 2

输出: [1,2]

示例 2:

输入: `nums = [1]`, k = 1

输出: [1]

💡 **学霸笔记:**

295. 数据流的中位数 【困难】

中位数是有序列表中间的数。如果列表长度是偶数，中位数则是中间两个数的平均值。

设计一个支持以下两种操作的数据结构：

- void addNum(int num) - 从数据流中添加一个整数到数据结构中。
- double findMedian() - 返回目前所有元素的中位数。

示例：

```
addNum(1)
addNum(2)
findMedian() -> 1.5
addNum(3)
findMedian() -> 2
```

📌 学霸笔记：

两个凡是，凡是考难题的就是为难你；凡是不考你 hot100 的就是不认真对待你

十四、贪心算法

本类共 4 道题

121. 买卖股票的最佳时机 【简单】

给定一个数组 prices，它的第 i 个元素 prices[i] 表示一支给定股票第 i 天的价格。

你只能选择 某一天 买入这只股票，并选择在 未来的某一个不同的日子 卖出该股票。设计一个算法来计算你能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 0 。

示例 1：

输入：[7,1,5,3,6,4]

输出：5

解释：在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = $6 - 1 = 5$ 。

示例 2：

输入：prices = [7,6,4,3,1]

输出：0

解释：在这种情况下，没有交易完成，所以最大利润为 0。

📌 学霸笔记：

55. 跳跃游戏 【中等】

给你一个非负整数数组 nums ，你最初位于数组的 第一个下标 。数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标，如果是，返回 true ； 否则，返回 false 。

示例 1：

输入：nums = [2,3,1,1,4]

输出：true

解释：可以先跳 1 步，从下标 0 到达下标 1，然后再从下标 1 跳 3 步到达最后一个下标。

示例 2：

输入: `nums = [3,2,1,0,4]`

输出: `false`

解释: 无论怎样, 总会到达下标为 3 的位置。但该下标的最大跳跃长度是 0 , 所以永远不可能到达最后一个下标。

📖 学霸笔记:

45. 跳跃游戏 II 【中等】

给定一个长度为 n 的 0 索引整数数组 `nums`。初始位置为 `nums[0]`。

每个元素 `nums[i]` 表示从索引 i 向前跳转的最大长度。换句话说, 如果你在 `nums[i]` 处, 你可以跳转到任意 `nums[i + j]` 处:

- $0 \leq j \leq \text{nums}[i]$

- $i + j < n$

返回到达 `nums[n - 1]` 的最小跳跃次数。生成的测试用例可以到达 `nums[n - 1]`。

示例 1:

输入: `nums = [2,3,1,1,4]`

输出: 2

解释: 跳到最后一个位置的最小跳跃数是 2。

示例 2:

输入: `nums = [2,3,0,1,4]`

输出: 2

📖 学霸笔记:

763. 划分字母区间 【中等】

给你一个字符串 s 。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。

注意，划分结果需要满足：将所有划分结果按顺序连接，得到的字符串仍然是 s 。

返回一个表示每个字符串片段的长度的列表。

示例 1：

输入： $s = \text{"ababcbacadefegdehijhklij"}$

输出： $[9,7,8]$

解释：

划分结果为 "ababcbaca" 、 "defegde" 、 "hijhklij" 。

示例 2：

输入： $s = \text{"eccbbbbbdec"}$

输出： $[10]$

📖 学霸笔记：

十五、动态规划

本类共 9 道题

70. 爬楼梯 【简单】

假设你正在爬楼梯。需要 n 阶你才能到达楼顶。

每次你可以爬 1 或 2 个台阶。你有多少种不同的方法可以爬到楼顶？

示例 1:

输入: $n = 2$

输出: 2

解释: 有两种方法可以爬到楼顶。

1. 1 阶 + 1 阶

2. 2 阶

示例 2:

输入: $n = 3$

输出: 3

解释: 有三种方法可以爬到楼顶。

1. 1 阶 + 1 阶 + 1 阶

2. 1 阶 + 2 阶

3. 2 阶 + 1 阶

📌 学霸笔记:

$Dp[i]$ 指 i 阶有几种方法爬到楼顶, 初始化 $dp[1] = 1, dp[2] = 2$ 开一层循环 i (3) - n

$Dp[i] = dp[i-1] + dp[i-2]$ 最后 return 结束战斗

```
class Solution {
    public int climbStairs(int n) {
        int[] dp = new int[n+2];
        dp[1] = 1;
        dp[2] = 2;
        for(int i = 3; i<=n;i++){
            dp[i] = dp[i-1] + dp[i-2];
        }
        return dp[n];
    }
}
```

118. 杨辉三角 【简单】

给定一个非负整数 numRows，生成「杨辉三角」的前 numRows 行。

示例 1:

输入: numRows = 5

输出: [[1],[1,1],[1,2,1],[1,3,3,1],[1,4,6,4,1]]

示例 2:

输入: numRows = 1

输出: [[1]]

💡 学霸笔记:

初始化 result, temp。temp add(1)，开两层循环，有模拟过程，外面 i-numRows,里面 j-i,里面先模拟下 add1，再 temp.add(result.get(i-1).get(j-2) + result.get(i-1).get(j-1)),再模拟 add1,result 加一下。结束战斗

```
class Solution {
    public List<List<Integer>> generate(int numRows) {
        List<List<Integer>> result = new ArrayList<>();
        List<Integer> temp = new ArrayList<>();
        temp.add(1);
        result.add(temp);
        if(numRows == 1){
            return result;
        }
        for(int i = 1;i<numRows;i++){
            temp = new ArrayList<>();
            temp.add(1);
            for(int j = 1;j < i;j++){
                temp.add(result.get(i - 1).get(j-1) + result.get(i - 1).get(j));
            }
            temp.add(1);
            result.add(temp);
        }
        return result;
    }
}
```

198. 打家劫舍 【中等】

你是一个专业的小偷，计划偷窃沿街的房屋。每间房内都藏有一定的现金，影响你偷窃的唯一制约因素就是相邻的房屋装有相互连通的防盗系统，如果两间相邻的房屋在同一晚上被小偷闯入，系统会自动报警。给定一个代表每个房屋存放金额的非负整数数组，计算你 不触动警报装置的情况下，一夜之内能够偷窃到的最高金额。

示例 1：

输入：[1,2,3,1]

输出：4

解释：偷窃 1 号房屋（金额 = 1），然后偷窃 3 号房屋（金额 = 3）。

示例 2：

输入：[2,7,9,3,1]

输出：12

📖 学霸笔记：

Dp[i] 指 i 天内偷的最多钱，dp[0] = 0,开一层循环 i(2)-n

dp[i] = Math.max(dp[i-1],dp[i - 2] + num[i-1]); 最后 return dp[n],结束战斗

```
class Solution {  
    public int rob(int[] nums) {  
        int[] dp = new int[nums.length + 666];  
        dp[0] = 0;  
        dp[1] = nums[0];  
        for(int i = 2;i <= nums.length;i++){  
            dp[i] = Math.max(dp[i-1],dp[i-2]+nums[i-1]);  
        }  
        return dp[nums.length];  
    }  
}
```

279. 完全平方数 【中等】

给你一个整数 n ，返回 和为 n 的完全平方数的最少数量。

完全平方数 是一个整数，其值等于另一个整数的平方；换句话说，其值等于一个整数自乘的积。例如，1、4、9 和 16 都是完全平方数，而 3 和 11 不是。

示例 1：

输入：n = 12

输出：3

解释：12 = 4 + 4 + 4

📖 学霸笔记：

Dp[i] 定义指平方数组成到 i 最小次数，定义 dp[0]=0,fill(...),开两层循环，外面 i(i*i <=n)-n,。里面 j(1)-n，里面判断 j >= i*i 就 dp[j] = Math.min(dp[j],dp[j - i*i] + 1) 最后 return dp[n]结束战斗（因为 1 的存在所以不用判断没结果）

```
class Solution {
    public int numSquares(int n) {
        int[] dp = new int[n+1];
        Arrays.fill(dp,Integer.MAX_VALUE);
        dp[0] = 0;
        for(int i = 1; i*i <= n;i++){
            for(int j = 1; j <= n; j++){
                if(j >= i*i){
                    dp[j] = Math.min(dp[j],dp[j - i*i] + 1);
                }
            }
        }
        return dp[n];
    }
}
```

322. 零钱兑换 【中等】

给你一个整数数组 coins，表示不同面额的硬币；以及一个整数 amount，表示总金额。

计算并返回可以凑成总金额所需的最少的硬币个数。如果没有任何一种硬币组合能组成总金额，返回 -1。

你可以认为每种硬币的数量是无限的。

示例 1：

输入：coins = [1, 5, 10, 25], amount = 41

输出：4

解释：25 + 10 + 5 + 1 = 41

示例 2：

输入 : **coins = [2], amount = 3**

输出 : **-1**

💡 学霸笔记 :

dp[i] 指组成 i 块钱的最少次数。记得定义 $dp[0] = 0$; `Arrays.fill(dp,Integer.MAX_VALUE)`

开两层循环, 外面 i(初始化 1)-coins, 里面 j(初始化 i)- 1,

$Dp[i] = \text{Math.min}(dp[i], dp[\text{coin} - j] + 1)$ 最后 `return dp[n]`, 结束战斗!

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 6];
        Arrays.fill(dp,Integer.MAX_VALUE - 1);
        dp[0] = 0;
        for(int coin : coins){
            for(int i = coin; i <= amount;i++){
                dp[i] = Math.min(dp[i],dp[i - coin] + 1);
            }
        }
        return dp[amount] == Integer.MAX_VALUE - 1? -1 : dp[amount];
    }
}
```

300. 最长递增子序列 【中等】

给你一个整数数组 `nums` , 找到其中最严格递增子序列的长度。

子序列 是由数组派生而来的序列, 删除 (或不删除) 数组中的元素而不改变其余元素的顺序。例如, `[3,6,2,7]` 是数组 `[0,3,1,6,2,2,7]` 的子序列。

进阶:

- 你能将算法的时间复杂度降低到 $O(n \log n)$ 吗?

示例 1:

输入: `nums = [10,9,2,5,3,7,101,18]`

输出: 4

解释: 最长递增子序列是 `[2,3,7,101]`, 因此长度为 4。

示例 2:

输入: `nums = [0,1,0,3,2,3]`

输出: 4

💡 学霸笔记:

152. 乘积最大子数组 【中等】

给你一个整数数组 `nums`，请你找出数组中乘积最大的非空连续子数组（该子数组中至少包含一个数字），并返回该子数组所对应的乘积。

测试用例的答案是一个 32-位 整数。

示例 1:

输入: `nums = [2,3,-2,4]`

输出: 6

解释: 子数组 `[2,3]` 有最大乘积 6。

示例 2:

输入: `nums = [-2,0,-1]`

输出: 0

解释: 结果不能为 2, 因为 `[-2,-1]` 不是连续子数组。

💡 学霸笔记:

416. 分割等和子集 【中等】

给你一个只包含正整数的非空数组 `nums`。请你判断是否可以将这个数组分割成两个子集，使得两个子集的元素和相等。

示例 1:

输入: `nums = [1,5,11,5]`

输出: `true`

解释: 数组可以分割成 `[1, 5, 5]` 和 `[11]`。

示例 2:

输入: `nums = [1,2,3,5]`

输出: `false`

解释: 数组不能分割成两个元素和相等的子集。

💡 学霸笔记:

32. 最长有效括号 【困难】

给你一个只包含 '(' 和 ')' 的字符串，找出最长有效（格式正确且连续）括号子串的长度。

示例 1:

输入: `s = "(()"`

输出: `2`

解释：最长有效括号子串是 "()"

示例 2:

输入: $s = ")()()"$

输出: 4

解释：最长有效括号子串是 "()()"

📖 学霸笔记:

两个凡是，凡是考难题的就是为难你；凡是不考你 hot100 的就是不认真对待你

十六、多维动态规划

本类共 5 道题

62. 不同路径 【中等】

一个机器人位于一个 $m \times n$ 网格的左上角（起始点在下图中标记为 "Start"）。

机器人每次只能向下或者向右移动一步。机器人试图达到网格的右下角（在下图中标记为 "Finish"）。

问总共有多少条不同的路径？

示例 1:

输入: $m = 3, n = 7$

输出: 28

示例 2:

输入: $m = 3, n = 2$

输出: 3

📖 学霸笔记:

64. 最小路径和 【中等】

给定一个包含非负整数的 $m \times n$ 网格 `grid`，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。

说明：每次只能向下或者向右移动一步。

示例 1:

输入: `grid = [[1,3,1],[1,5,1],[4,2,1]]`

输出: 7

解释: 因为路径 `1→3→1→1→1` 的总和最小。

示例 2:

输入: `grid = [[1,2,3],[4,5,6]]`

输出: 12

📖 学霸笔记:

5. 最长回文子串 【中等】

给你一个字符串 `s`，找到 `s` 中最长的回文子串。

示例 1:

输入：**s = "babad"**

输出：**"bab"**

解释：**"aba"** 同样是符合题意的答案。

示例 2:

输入：**s = "cbbd"**

输出：**"bb"**

📖 学霸笔记:

1143. 最长公共子序列 【中等】

给定两个字符串 text1 和 text2，返回这两个字符串的最长 公共子序列 的长度。如果不存在 公共子序列，返回 0。

一个字符串的 子序列 是指这样一个新的字符串：它是由原字符串在不改变字符的相对顺序的情况下删除某些字符（也可以不删除任何字符）后组成的新字符串。

例如，"ace" 是 "abcde" 的子序列，但 "aec" 不是 "abcde" 的子序列。

两个字符串的 公共子序列 是这两个字符串所共同拥有的子序列。

示例 1：

输入：**text1 = "abcde", text2 = "ace"**

输出：**3**

解释：最长公共子序列是 **"ace"**，它的长度为 **3**。

示例 2：

输入：**text1 = "abc", text2 = "abc"**

输出：3

📌 学霸笔记：

72. 编辑距离 【中等】

给你两个单词 word1 和 word2，请返回将 word1 转换成 word2 所使用的最少操作数。

你可以对一个单词进行如下三种操作：

- 插入一个字符
- 删除一个字符
- 替换一个字符

示例 1：

输入：word1 = "horse", word2 = "ros"

输出：3

解释：

horse -> rorse (将 'h' 替换为 'r')

rorse -> rose (删除 'r')

rose -> ros (删除 'e')

示例 2：

输入：word1 = "intention", word2 = "execution"

输出：5

📌 学霸笔记：

十七、技巧

本类共 5 道题

136. 只出现一次的数字 【简单】

给你一个 非空 整数数组 `nums` ，除了某个元素只出现一次以外，其余每个元素均出现两次。找出那个只出现了一次的元素。

你必须设计并实现线性时间复杂度的算法来解决此问题，且该算法只使用常量额外空间。

示例 1：

输入：`nums = [2,2,1]`

输出：`1`

示例 2：

输入：`nums = [4,1,2,1,2]`

输出：`4`

📖 学霸笔记：

169. 多数元素 【简单】

给定一个大小为 `n` 的数组 `nums` ，返回其中的多数元素。多数元素是指在数组中出现次数大于 $\lfloor n/2 \rfloor$ 的元素。

你可以假设数组是非空的，并且给定的数组总是存在多数元素。

示例 1：

输入: `nums = [3,2,3]`

输出: 3

示例 2:

输入: `nums = [2,2,1,1,1,2,2]`

输出: 2

📌 学霸笔记:

75. 颜色分类 【中等】

给定一个包含红色、白色和蓝色、共 n 个元素的数组 `nums`，原地对它们进行排序，使得相同颜色的元素相邻，并按照红色、白色、蓝色顺序排列。

我们使用整数 0、1 和 2 分别表示红色、白色和蓝色。

必须在不使用库内置的 `sort` 函数的情况下解决这个问题。

示例 1:

输入: `nums = [2,0,2,1,1,0]`

输出: `[0,0,1,1,2,2]`

示例 2:

输入: `nums = [2,0,1]`

输出: `[0,1,2]`

📌 学霸笔记:

31. 下一个排列 【中等】

整数数组的一个排列就是将其所有成员以序列或线性顺序排列。

- 例如， $arr = [1,2,3]$ ，以下这些都可以视作 arr 的排列： $[1,2,3]$ 、 $[1,3,2]$ 、 $[3,1,2]$ 、 $[2,3,1]$ 。

整数数组的下一个排列是指其整数的下一个字典序更大的排列。更正式地，如果数组的所有排列根据其字典顺序从小到大排列在一个容器中，那么数组的下一个排列就是在这个有序容器中排在它后面的那个排列。如果不存在下一个更大的排列，那么这个数组必须重排为字典序最小的排列（即，其元素按升序排列）。

- 例如， $arr = [1,2,3]$ 的下一个排列是 $[1,3,2]$ 。

- 类似地， $arr = [2,3,1]$ 的下一个排列是 $[3,1,2]$ 。

- 而 $arr = [3,2,1]$ 的下一个排列是 $[1,2,3]$ ，因为 $[3,2,1]$ 不存在一个字典序更大的排列。

给你一个整数数组 $nums$ ，找出 $nums$ 的下一个排列。

必须原地修改，只允许使用额外常数空间。

示例 1：

输入： $nums = [1,2,3]$

输出： $[1,3,2]$

示例 2：

输入： $nums = [3,2,1]$

输出： $[1,2,3]$

📌 学霸笔记:

287. 寻找重复数 【中等】

给定一个包含 $n + 1$ 个整数的数组 `nums`，其数字都在 $[1, n]$ 范围内（包括 1 和 n ），可知至少存在一个重复的整数。

假设 `nums` 只有一个重复的整数，返回 这个重复的数。

你设计的解决方案必须 不修改 数组 `nums` 且只用常量级 $O(1)$ 额外空间。

示例 1:

输入: `nums = [1,3,4,2,2]`

输出: 2

示例 2:

输入: `nums = [3,1,3,4,2]`

输出: 3

📌 学霸笔记:

附录：力扣精彩评论

1：我是一个 SB

2：woshisb

3：有人香车宝马，有人天赋异禀，还有人在力扣做题，只会看题解。

4：美团拼好饭专送一面

5：碧桂园保安一面

6：塘心男演员一面

7：吉野家店员一面

8：蜜雪冰城二面

9：感觉自己好没用

10：执行通过笑嘻嘻，一看击败百分七

11：有人相爱，有人在深夜开车去看海，有人 LeetCode 第一题做不出来

12：八嘎，这么难

13：玩偶姐姐三面，柚子猫一面，刘玥二面，娜娜二面



清浅

2026.04.26

杀

公司工位固定 k 个。

每天来一个新人。

他从队伍末尾往前杀：见一个比他弱的，杀一个。

杀完，自己坐到最后。

如果人数超过 k ，就把最前面那个人杀了，不管他强不强。

然后，记下现在最前面那个人的名字。

日复一日。

一年后翻开本子——

上面记的，是每个 k 天窗口里，活到最后的那把刀。

这个世界从不记得谁活最久，只记得每个窗口滑过时，最前面那个杀红了眼的人。

14.:

15：我是区

16：千帆过尽皆不是，轻舟已过万重山。循环遍历，我遇见更好，我也感谢曾遇，好湿好湿

